```
      expect(frequentFlyer.status_points).to eq(100)
    end

    it 'should upgrade member status when enough points are earned' do
      frequentFlyer = FrequentFlyer.new

      frequentFlyer.earn_status_points(100).earn_status_points(200)

      expect(frequentFlyer.status).to eq('SILVER')
    end
  end
end
```

**Contexts can contain several related requirements.**

❶ **Given...**

❷ **When...**

❸ **Then...**

Specifications written this way should stay simple and concise, respecting the BDD approach of favoring simplicity and communication. If you study the structure of these specifications, you may notice that they actually respect a "Given ... When ... Then" style. For example, in the last specification above, the *Given* creates a new Frequent Flyer object ❶, the *When* is the action you're demonstrating ❷, and the *Then* is the expected outcome ❸. Even in the more concise RSpec style, respecting this structure will make your specifications much cleaner and easier to understand.

These specifications are not only quite readable, they'll also produce reports that describe the low-level specifications in a more structured way than a list of unit-test results. For example, running `rspec` from the command line would produce a summary of the requirements in text form:

```
$rspec --format documentation

FrequentFlyer
  when the frequent flyer account is first created
    should initially have Bronze status
  when a new member starts to fly with Flying High
    should earn points for each flight
    should upgrade member status when enough points are earned
```

RSpec has many other interesting features that we don't have time to discuss here. Since RSpec was released, other languages have also adopted RSpec-style testing libraries for BDD unit testing, including NSpec for .NET and a number of libraries for JavaScript. As an example, let's look at one of these.

#### Executable specifications in JavaScript with Jasmine

JavaScript is playing an increasingly important role in modern application development, both on the client side, and, with the rise of JavaScript application platforms like Node.js, on the server side.

Although more traditional unit-testing tools such as QUnit are widely used in the JavaScript world, there are several BDD-style JavaScript unit-testing libraries around. The best known of these are Jasmine (http://jasmine.github.io) and Mocha (http://visionmedia.github.io/mocha/). As far as BDD goes, both these libraries have a very similar style. You've already seen Jasmine in action in chapter 9, where you used it to test an AngularJS web application. Let's take a closer look at how it works.